Introduction to Software Systems
COMP 206

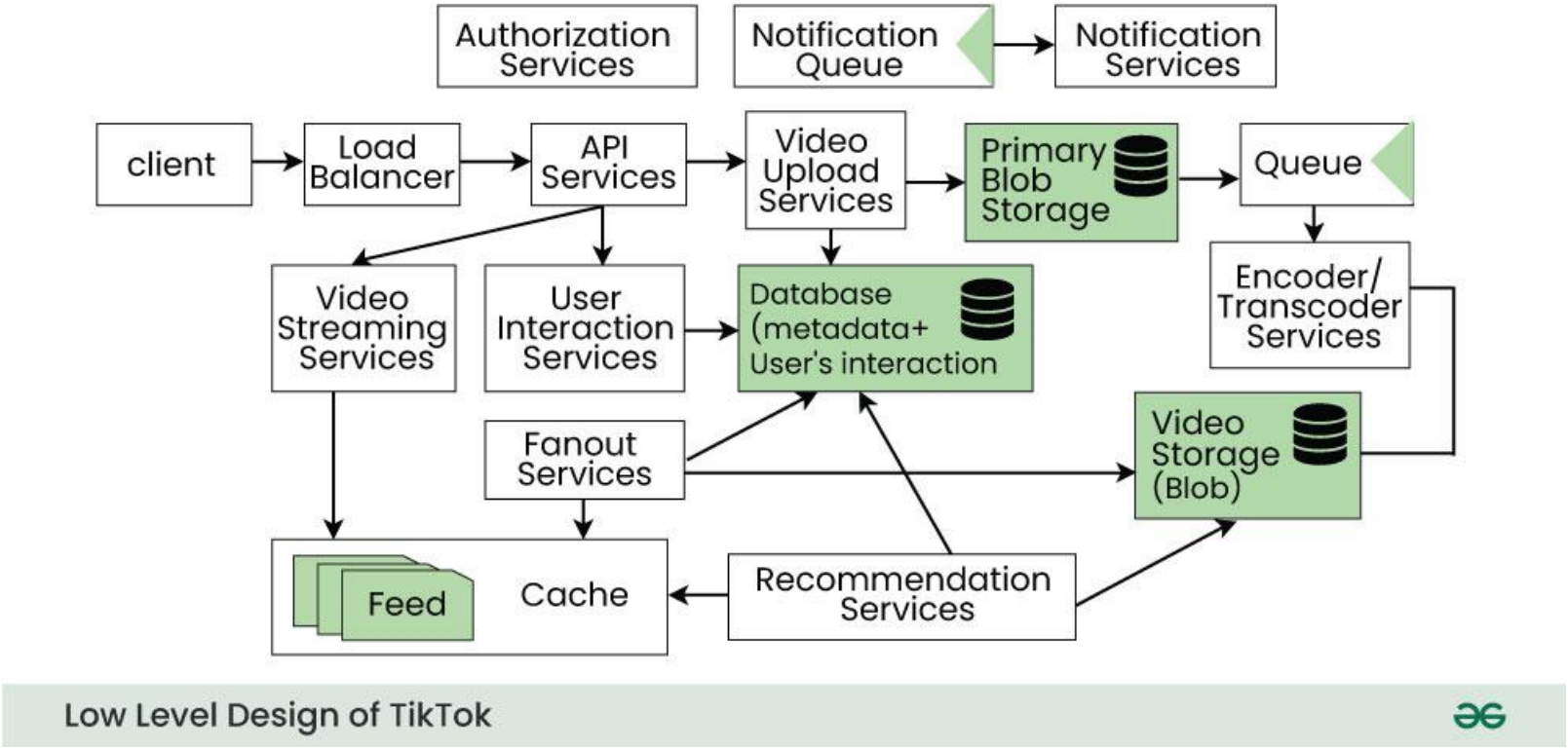
Instructors: Jacob Errington & Max Kopinsky
School of Computer Science
McGill University

Lesson 0: Welcome to COMP 206

Software Systems...

What do you think it takes to build something like Instagram, Snapchat, or TikTok?

What do you think it takes to build something like Instagram, Snapchat, or TikTok?



So that's a software system

"A set of *interconnected*, *communicating* software artifacts, together with their specifications, configuration files, and documentation is called a *software system*."

Instructor – Jacob Thomas Errington

- Born & raised in Montreal ♡
- B.Sc. Math & CS @ McGill
- M.Sc. CS @ McGill - proof theory
- Worked in fintech; didn't work for me
- Faculty Lecturer @ McGill since 2022

Fun. Breadmaking, video games, bodybuilding, chess, coding.

Instructor – Max Kopinsky

- Originally from Chicago
- B.Sc. Math & CS @ UIUC
- Worked one year in Formal Verification
- M.Sc. CS @ McGill - type error provenance
- Faculty Lecturer @ McGill since 2024

Fun. Video & board games (find me at BGSS), programming. Always carrying cards!

A coarse-grain view of the course

Module 1. Getting to grips with the command-line.

A coarse-grain view of the course

Module 1. Getting to grips with the command-line.

Module 2. Programming in C.

A coarse-grain view of the course

Module 1. Getting to grips with the command-line.

Module 2. Programming in C.

Module 3. Cool stuff in systems engineering: cybersecurity, the internet.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.
- Write bash scripts to automate tasks relating to software development.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.
- Write bash scripts to automate tasks relating to software development.
- Use git to track modifications to bodies of source code.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.
- Write bash scripts to automate tasks relating to software development.
- Use git to track modifications to bodies of source code.
- Write robust system interactions in C.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.
- Write bash scripts to automate tasks relating to software development.
- Use git to track modifications to bodies of source code.
- Write robust system interactions in C.
- Use appropriate algorithms and data structures in C.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.
- Write bash scripts to automate tasks relating to software development.
- Use git to track modifications to bodies of source code.
- Write robust system interactions in C.
- Use appropriate algorithms and data structures in C.
- Articulate basic security vulnerabilities of C programs.

A finer-grain view: the learning goals

By the end of this course, you will be able to:

- Use the command-line on Linux to read, write, and manage files.
- Compose atomic command-line programs into pipelines.
- Write bash scripts to automate tasks relating to software development.
- Use git to track modifications to bodies of source code.
- Write robust system interactions in C.
- Use appropriate algorithms and data structures in C.
- Articulate basic security vulnerabilities of C programs.
- Explain in broad strokes the communication principles of the Internet.

Why obtain these skills? Why take this course?

- Boring reason:
  it's a required course in any CS program.

Why obtain these skills? Why take this course?

- Boring reason:
  it's a required course in any CS program.
- Practical reason:
  you will need the command-line sooner or later!

Why obtain these skills? Why take this course?

- Boring reason:
  it's a required course in any CS program.
- Practical reason:
  you will need the command-line sooner or later!
- Efficiency reason:
  for performance, you sometimes need low-level programming

Why obtain these skills? Why take this course?

- Boring reason:
  it's a required course in any CS program.
- Practical reason:
  you will need the command-line sooner or later!
- Efficiency reason:
  for performance, you sometimes need low-level programming
- Philosophical reason:
  CS is the science and art of *abstraction*.

A concrete view: the gist of the syllabus

Prereqs. COMP 202 or COMP 250.

Coreq. COMP 273.

A concrete view: the gist of the syllabus

Prereqs. COMP 202 or COMP 250.

Coreq. COMP 273.

Assessment.

- Homework: $10 \times 2\% = 20\%$
- In-class participation: 10%
- Midterms: $2 \times 15\% = 30\%$
- Quiz: 10%
- Final: 30%

Special policies

Flexible homework due dates. Soft deadline: end of the week.
Hard deadline: the associated midterm.

Special policies

Flexible homework due dates. Soft deadline: end of the week. Hard deadline: the associated midterm.

Homework are pass/fail. Submission passes all automated test cases to achieve the marks.

Special policies

Flexible homework due dates. Soft deadline: end of the week. Hard deadline: the associated midterm.

Homework are pass/fail. Submission passes all automated test cases to achieve the marks.

Optional final exam. The final exam serves as a re-do of one or both of the midterms, with a small penalty.

Special policies

Flexible homework due dates. Soft deadline: end of the week. Hard deadline: the associated midterm.

Homework are pass/fail. Submission passes all automated test cases to achieve the marks.

Optional final exam. The final exam serves as a re-do of one or both of the midterms, with a small penalty.

Modules 1 and 2 are must-pass. A failing ($< 55\%$) grade in the exam component for module 1 or 2 will cap your final letter grade at a D.

Teaching methods

Lessons will generally consist of the following.

- Pre-class videos about certain topics.

Teaching methods

Lessons will generally consist of the following.

- Pre-class videos about certain topics.
- Lecturing about theoretical ideas.

Teaching methods

Lessons will generally consist of the following.

- Pre-class videos about certain topics.
- Lecturing about theoretical ideas.
- Live-coding demos of practical concepts.
  You'll want to follow along on your laptop for these!

Teaching methods

Lessons will generally consist of the following.

- Pre-class videos about certain topics.
- Lecturing about theoretical ideas.
- Live-coding demos of practical concepts.
  You'll want to follow along on your laptop for these!
- Small group discussions about prompts I give you.

Teaching methods

Lessons will generally consist of the following.

- Pre-class videos about certain topics.
- Lecturing about theoretical ideas.
- Live-coding demos of practical concepts.
  You'll want to follow along on your laptop for these!
- Small group discussions about prompts I give you.
- In-class coding exercises.

Teaching methods

Lessons will generally consist of the following.

- Pre-class videos about certain topics.
- Lecturing about theoretical ideas.
- Live-coding demos of practical concepts.
  You'll want to follow along on your laptop for these!
- Small group discussions about prompts I give you.
- In-class coding exercises.
- Polls in class.

Teaching assistants

We have 16 fantastic teaching assistants this year.

What does a TA do?

TAs:

- Grade tests.
- Hold labs.

What does a TA do?

TAs:

- Grade tests.
- Hold labs.
- Hold office hours – go to them! It's free tutoring!

What does a TA do?

TAs:

- Grade tests.
- Hold labs.
- Hold office hours – go to them! It's free tutoring!
- Answer questions online – ask questions!

Labs

- Walk through a demo for 15 minutes.
- You do an exercise for about 15 minutes.
- TA solves the problem and Q & A for 15 minutes.

Linux Guru Challenge

- For those of you who want an extra challenge, there will be three additional, more difficult assignments.
- Completing these will give you a small bonus to your final course grade.
- These are graded on a pass/fail basis, through a brief interview with a TA.

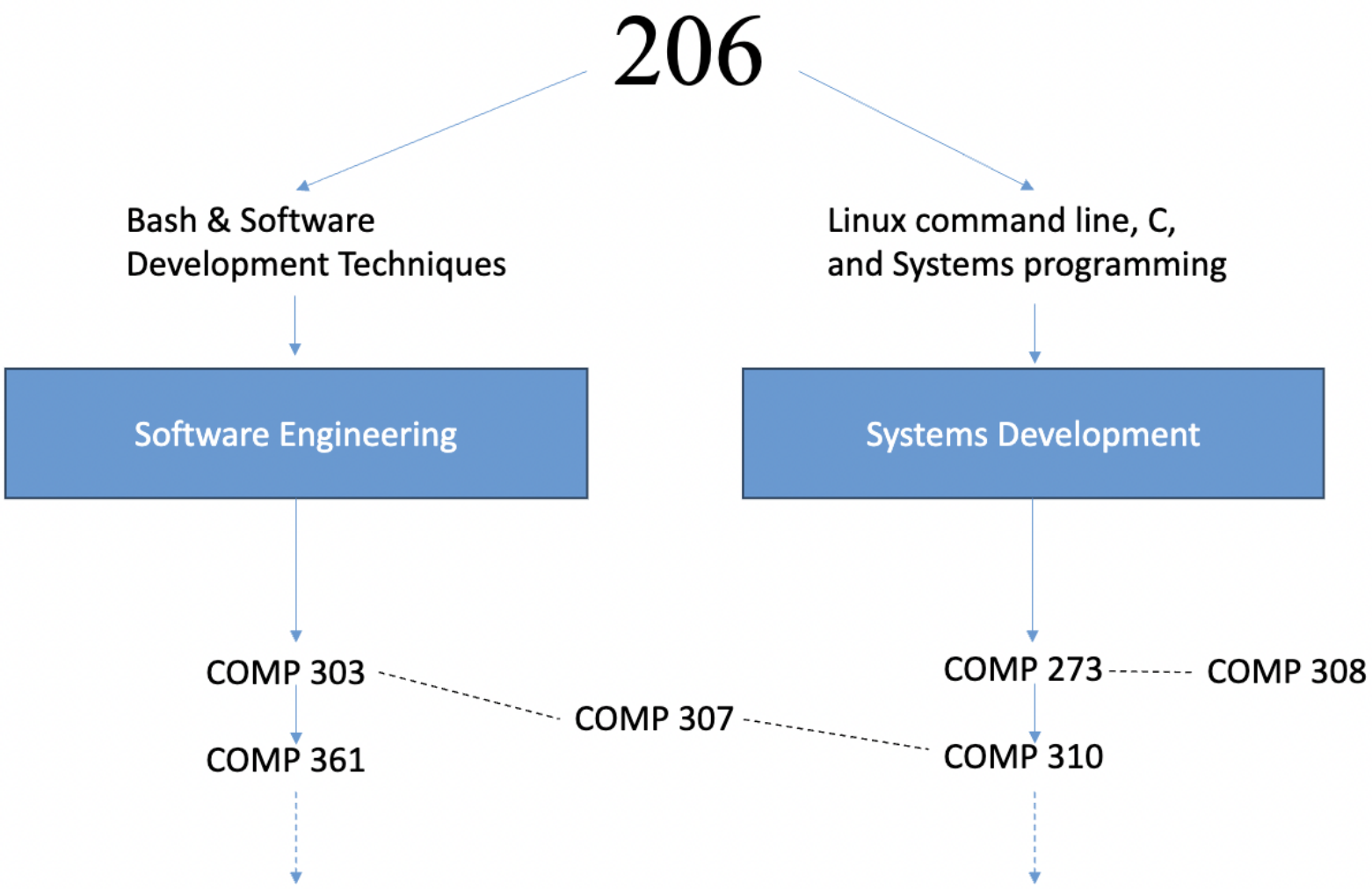
Linux Guru Challenge

- For those of you who want an extra challenge, there will be three additional, more difficult assignments.
- Completing these will give you a small bonus to your final course grade.
- These are graded on a pass/fail basis, through a brief interview with a TA.

Number of challenges completed	Bonus to your grade
1	0.5
2	1
3	1.5

How does this course fit in?

How does this course fit in?



Ambitious goals

This is not an easy course...
Expectations are high!

How can you succeed in this course?

How can you succeed in this course?

- Come to class. (Eat beforehand!)

How can you succeed in this course?

- Come to class. (Eat beforehand!)
- Participate in class: coding problems, discussions, ask questions

How can you succeed in this course?

- Come to class. (Eat beforehand!)
- Participate in class: coding problems, discussions, ask questions
- Participate outside class: TA office hours, Discord, instructor office hours
  1-on-1 is way better than 1-on-600!

How can you succeed in this course?

- Come to class. (Eat beforehand!)
- Participate in class: coding problems, discussions, ask questions
- Participate outside class: TA office hours, Discord, instructor office hours
  1-on-1 is way better than 1-on-600!
- Do the homework yourself, without the use of AI!

How can you succeed in this course?

- Come to class. (Eat beforehand!)
- Participate in class: coding problems, discussions, ask questions
- Participate outside class: TA office hours, Discord, instructor office hours
  1-on-1 is way better than 1-on-600!
- Do the homework yourself, without the use of AI!
- (Sadly) practice coding on paper.

How can we help you to succeed?

How can we help you to succeed?

- Interactive lessons.

How can we help you to succeed?

- Interactive lessons.
- Prompt feedback.

How can we help you to succeed?

- Interactive lessons.
- Prompt feedback.
- Learning names.

How can we help you to succeed?

- Interactive lessons.
- Prompt feedback.
- Learning names.
- Patience.

Communication outside of class

We will be using Discord for online discussion:
`https://discord.gg/3QB9kmgRdC`

TAs on Discord are assigned a "TA" role and their names appear in red.